RAJALAKSHMI ENGINEERING COLLEGE

Arnold E 250701075-2025-CSE

Dashboard My courses

CS23331-Design and Analysis of Algorithms-2025 Batch-CSE, IT, AIML & AIDS

Course Participants Grades Competencies

General

Collapse all

Finding Time Complexity of Algorithms

Problem 1 Finding Complexity using Counter Method — Done

Opened: Wednesday, 16 July 2025, 8:00 AM
Closes: Saturday, 11 October 2025, 11:59 PM

Problem 2 Finding Complexity using Counter method — Done

Opened: Wednesday, 16 July 2025, 9:07 AM
Closes: Saturday, 11 October 2025, 11:59 PM

Problem 3 Finding Complexity using Counter Method — Done

Opened: Tuesday, 15 July 2025, 10:07 AM
Closes: Saturday, 11 October 2025, 11:59 PM

Problem 4 Finding Complexity using Counter Method — Done

Opened: Tuesday, 15 July 2025, 10:56 AM
Closes: Saturday, 11 October 2025, 11:59 PM

Problem 5 Finding Complexity using counter method — Done

Opened: Tuesday, 15 July 2025, 10:58 AM
Closes: Saturday, 11 October 2025, 11:59 PM

Greedy Algorithms

1-Coin Problem — Done

Opened: Monday, 21 July 2025, 10:00 AM
Closes: Saturday, 20 October 2025, 11:59 AM

2-B-Cookies Problem — Done

Opened: Wednesday, 15 July 2026, 10:00 AM
Closes: Tuesday, 20 October 2026, 11:56 AM

3-G-Burger Problem — Done

Opened: Wednesday, 15 July 2026, 10:00 AM
Closes: Tuesday, 20 October 2026, 11:56 AM

4-G-Array Sum max problem — Done

Opened: Saturday, 15 August 2026, 10:00 AM
Closes: Tuesday, 20 October 2026, 11:57 AM

5-G-Product of Array elements-Minimum — Done

Opened: Wednesday, 15 July 2026, 10:00 AM
Closes: Tuesday, 20 October 2026, 11:57 AM

Divide and Conquer

1-Number of Zeros in a Given Array — Done

Opened: Tuesday, 21 July 2026, 11:34 AM
Closes: Tuesday, 20 October 2026, 11:54 AM

2-Majority Element — Done

Opened: Monday, 20 July 2026, 11:17 AM
Closes: Tuesday, 20 October 2026, 3:13 PM

3-Finding Floor Value — Done

Opened: Sunday, 19 July 2026, 12:17 PM
Closes: Tuesday, 20 October 2026, 3:14 PM

4-Two Elements sum to x — Done

Opened: Monday, 20 July 2026, 12:17 PM
Closes: Tuesday, 20 October 2026, 3:14 PM

5-Implementation of Quick Sort — Done

Opened: Thursday, 30 July 2026, 9:49 AM
Closes: Tuesday, 20 October 2026, 3:15 PM

Dynamic Programming

1-DP-Playing with Numbers — Done

Opened: Tuesday, 1 July 2025, 12:27 AM
Closes: Friday, 30 October 2026, 8:19 AM

2-DP-Playing with chessboard — Done

Opened: Friday, 11 September 2026, 8:22 AM
Closes: Friday, 30 October 2026, 8:22 AM

3-DP-Longest Common Subsequence — Done

Opened: Tuesday, 1 July 2025, 12:17 AM
Closes: Friday, 30 October 2026, 8:23 AM

4-DP-Longest non-decreasing Subsequence — Done

Opened: Tuesday, 1 July 2025, 12:03 AM
Closes: Saturday, 31 October 2026, 8:23 AM

Competitive Programming

1-Finding Duplicates-O(n^2) Time Complexity,O(1) Space Complexity — Done

Closes: Sunday, 11 October 2026, 8:24 AM

2-Finding Duplicates-O(n) Time Complexity,O(1) Space Complexity — Done

Closes: Sunday, 11 October 2026, 8:25 AM

3-Print Intersection of 2 sorted arrays-O(m*n)Time Complexity,O(1) Space Complexity — Done

Closes: Sunday, 11 October 2026, 8:25 AM

4-Print Intersection of 2 sorted arrays-O(m+n)Time Complexity,O(1) Space Complexity — Done

Closes: Sunday, 11 October 2026, 8:25 AM

5-Pair with Difference-O(n^2)Time Complexity,O(1) Space Complexity — Done

Closes: Sunday, 11 October 2026, 8:26 AM

6-Pair with Difference -O(n) Time Complexity,O(1) Space Complexity — Done

Closes: Sunday, 11 October 2026, 8:26 AM

Problem 1: Finding Complexity using Counter Method

Started on	Monday, 17 August 2026, 1:35 PM
State	Finished
Completed on	Monday, 17 August 2026, 1:57 PM
Time taken	22 mins 36 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void function (int n)
{
    int i= 1;

    int s =1;

    while(s <= n)
    {
        i++;
        s += i;
     }
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

For example:

Input	Result
9	12

Answer: (penalty regime: 0 %)

Ace editor not ready. Perhaps reload page?

Falling back to raw text area.

```
void function(int n) {
    int i = 1;
    int s = 1;
    int count = 0;
    count += 2;
    while (s <= n) {
        count++;
        i++;
        count++;
        s += i;
        count++;
```

Quiz navigation

1

Finish review

```
    }
    count++;
    printf("%d\n", count);
}

int main() {
    int n;
```

	Input	Expected	Got	
✔	9	12	12	✔
✔	4	9	9	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Finish review

Back to Course

Problem 2: Finding Complexity using Counter method

Started on	Monday, 17 August 2026, 2:04 PM
State	Finished
Completed on	Monday, 17 August 2026, 2:09 PM
Time taken	5 mins 12 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Convert the following algorithm into a program and find its time complexity using the counter method.

```
void func(int n)
{
    if(n==1)
    {
      printf("*");
    }
    else
    {
     for(int i=1; i<=n; i++)
     {
       for(int j=1; j<=n; j++)
       {
          printf("*");
          printf("*");
          break;
       }
     }
   }
 }
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

Answer: (penalty regime: 0 %)

```
#include <stdio.h>
void func(int n) {
    int count = 0;
    if (n == 1) {
        count++;
    } else {
        count++;
        for (int i = 1; i <= n; i++) {
            count++;
            for (int j = 1; j <= n; j++) {
                count++;
                count++;
                count++;
                break;
```

Quiz navigation

1

Finish review

```
            }
            count++;
        }
        count++;
    }
    printf("%d\n", count);
}
int main() {
    int n;
    scanf("%d", &n);
    func(n);
    return 0;
}
```

	Input	Expected	Got	
✔	2	12	12	✔
✔	1000	5002	5002	✔
✔	143	717	717	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Finish review

Back to Course

Problem 3: Finding Complexity using Counter Method

Started on	Tuesday, 29 September 2026, 10:26 AM
State	Finished
Completed on	Tuesday, 29 September 2026, 10:25 PM
Time taken	11 hours 59 mins
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Convert the following algorithm into a program and find its time complexity using counter method.

```
Factor(num) {
{
   for (i = 1; i <= num;++i)
   {
    if (num % i== 0)
       {
         printf("%d ", i);
       }
    }
 }
```

Note: No need of counter increment for declarations and scanf() and counter variable printf() statement.

Input:

A positive Integer n

Output:

Print the value of the counter variable

Answer:

```c
#include <stdio.h>

int main() {
    int num, i;
    int count = 0;

    // No counter increment for declarations and scanf
    scanf("%d", &num);

    // Autograder ignores initialization (i=1) and increment (++i)
    for (i = 1; i <= num; ++i) {

        count++; // 1 step for loop condition check (true cases)

        count++; // 1 step for if condition
        if (num % i == 0) {
            // Factor printf is commented out because the expected output
            // strictly expects ONLY the final counter value.
            // printf("%d ", i);

            count++; // 1 step for the execution inside the if-block
        }
    }
```

Quiz navigation

1

Finish review

```
    count++; // 1 step for the final loop condition check (false case)

    // Print only the final counter value
    printf("%d\n", count);

    return 0;
}
```

	Input	Expected	Got	
✔	12	31	31	✔
✔	25	54	54	✔
✔	4	12	12	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Finish review

Back to Course

Problem 4: Finding Complexity using Counter Method

Started on	Tuesday, 29 September 2026, 10:26 PM
State	Finished
Completed on	Tuesday, 29 September 2026, 10:33 PM
Time taken	7 mins 8 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Convert the following algorithm into a program and find its time

complexity using counter method.

```
void function(int n)
{
    int c= 0;
    for(int i=n/2; i<n; i++)
        for(int j=1; j<n; j = 2 * j)
            for(int k=1; k<n; k = k * 2)
                c++;
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

Answer:

```c
#include <stdio.h>

int main() {
    int n;
    int count = 0;
    int c = 0;

    scanf("%d", &n);

    count++;

    for (int i = n / 2; i < n; i++) {
        count++;

        for (int j = 1; j < n; j = 2 * j) {
            count++;

            for (int k = 1; k < n; k = k * 2) {
                count++;
                c++;
                count++;
            }
            count++;
        }
```

```
        count++;
    }
    count++;

    printf("%d\n", count);

    return 0;
}
```

	Input	Expected	Got	
✔	4	30	30	✔
✔	10	212	212	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Finish review

Back to Course

Data retention summary

Problem 5: Finding Complexity using counter method

Started on	Tuesday, 29 September 2026, 10:34 PM
State	Finished
Completed on	Tuesday, 29 September 2026, 10:35 PM
Time taken	1 min
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Convert the following algorithm into a program and find its time complexity using counter method.

```
void reverse(int n)
{
   int rev = 0, remainder;
   while (n != 0)
    {
        remainder = n % 10;
        rev = rev * 10 + remainder;
        n/= 10;

    }
print(rev);
}
```

Note: No need of counter increment for declarations and scanf() and count variable printf() statements.

Input:

A positive Integer n

Output:

Print the value of the counter variable

Answer:

```c
#include <stdio.h>

int main() {
    int n;
    int count = 0;
    int rev = 0, remainder;

    scanf("%d", &n);

    count++;

    while (n != 0) {
        count++;

        remainder = n % 10;
        count++;

        rev = rev * 10 + remainder;
        count++;

        n /= 10;
        count++;
```

Quiz navigation

1

Finish review

```
    }
    count++;

    count++;

    printf("%d\n", count);

    return 0;
}
```

	Input	Expected	Got	
✔	12	11	11	✔
✔	1234	19	19	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Finish review

Back to Course

1-G-Coin Problem

Started on	Friday, 28 August 2026, 11:50 AM
State	Finished
Completed on	Friday, 28 August 2026, 11:55 AM
Time taken	5 mins 33 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Write a program to take value V and we want to make change for V Rs, and we have infinite supply of each of the denominations in Indian currency, i.e., we have infinite supply of { 1, 2, 5, 10, 20, 50, 100, 500, 1000} valued coins/notes, what is the minimum number of coins and/or notes needed to make the change.

Input Format:

Take an integer from stdin.

Output Format:

print the integer which is change of the number.

Example Input :

64

Output:

4

Explanaton:

We need a 50 Rs note and a 10 Rs note and two 2 rupee coins.

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
int main() {
    int V;
    scanf("%d", &V);
    int denominations[] = {1000, 500, 100, 50, 20, 10, 5, 2, 1};
    int n = sizeof(denominations) / sizeof(denominations[0]);
    int count = 0;
    for (int i = 0; i < n; i++) {
        if (V >= denominations[i]) {
            count += V / denominations[i];
            V = V % denominations[i];
        }
    }
    printf("%d\n", count);
    return 0;
}
```

	Input	Expected	Got	
✔	49	5	5	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Finish review

Back to Course

2-G-Cookies Problem

Started on	Sunday, 30 August 2026, 6:59 PM
State	Finished
Completed on	Sunday, 30 August 2026, 8:16 PM
Time taken	1 hour 16 mins
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 Correct Mark 1.00 out of 1.00 Flag question

Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.

Each child i has a greed factor g[i], which is the minimum size of a cookie that the child will be content with; and each cookie j has a size s[j]. If s[j] >= g[i], we can assign the cookie j to the child i, and the child i will be content. Your goal is to maximize the number of your content children and output the maximum number.

Example 1:

Input:

3

1 2 3

2

1 1

Output:

1

Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3.

And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content.

You need to output 1.

Constraints:

1 <= g.length <= 3 * 10^4

0 <= s.length <= 3 * 10^4

1 <= g[i], s[j] <= 2^31 - 1

Answer: (penalty regime: 0 %)

```c
#include<stdio.h>
#include<stdlib.h>
int cmp(const void*a,const void *b){
    return(*(int*)a-*(int*)b);
}
int main(){
    int n,m;
    scanf("%d",&n);
    int g[n];
    for(int i=0;i<n;i++){
        scanf("%d",&g[i]);
    }
    scanf("%d",&m);
    int s[m];
    for(int j=0;j<m;j++)
```

```
    {
        scanf("%d",&s[j]);
    }
    qsort(g,n,sizeof(int),cmp);
    qsort(s,m,sizeof(int),cmp);
    int i=0,j=0,count=0;
    while(i<n && j<m){
        if(s[j]>=g[i]){
            count++;
            i++;
            j++;
        }
        else{
            j++;
        }
    }
    printf("%d\n",count);
    return 0;
}
```

	Input	Expected	Got	
✔	2 1 2 3 1 2 3	2	2	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Finish review

Back to Course

Data retention summary

3-G-Burger Problem

Started on	Tuesday, 29 September 2026, 10:35 PM
State	Finished
Completed on	Tuesday, 29 September 2026, 10:36 PM
Time taken	48 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

A person needs to eat burgers. Each burger contains a count of calorie. After eating the burger, the person needs
If he has eaten i burgers with c calories each, then he has to run at least $3^i * c$ kilometers to burn out the cal
burgers with the count of calorie in the order: [1, 3, 2], the kilometers he needs to run are $(3^0 * 1) + (3^1 * 3)$
But this is not the minimum, so need to try out other orders of consumption and choose the minimum value. Determi
he needs to run. Note: He can eat burger in any order and use an efficient sorting algorithm.Apply greedy approac

Input Format

First Line contains the number of burgers

Second line contains calories of each burger which is n space-separate integers

Output Format

Print: Minimum number of kilometers needed to run to burn out the calories

Sample Input

```
3
5 10 7
```

Sample Output

```
76
```

For example:

Test	Input	Result
Test Case 1	3 1 3 2	18

Answer: (penalty regime: 0 %)

```c
#include<stdio.h>
#include<math.h>
#include<stdlib.h>

int cmp(const void *a, const void *b) {
    return (*(int*)b - *(int*)a);
}

int main(){
    int n,cnt=0;
    scanf("%d",&n);
    int d[n];
    for(int i=0;i<n;i++){
        scanf("%d",&d[i]);
```

Quiz navigation

1

Finish review

```
    }
    qsort(d,n,sizeof(int),cmp);
    for(int i=0;i<n;i++){
        cnt += pow(n,i) * d[i];
    }
    printf("%d",cnt);
    return 0;
}
```

	Test	Input	Expected	Got	
✔	Test Case 1	3 1 3 2	18	18	✔
✔	Test Case 2	4 7 4 9 6	389	389	✔
✔	Test Case 3	3 5 10 7	76	76	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Finish review

Back to Course

4-G-Array Sum max problem

Started on	Sunday, 30 August 2026, 9:14 PM
State	Finished
Completed on	Sunday, 30 August 2026, 9:18 PM
Time taken	4 mins 11 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Given an array of N integer, we have to maximize the sum of arr[i] * i, where i is the index of the element (i = 0, 1, 2, ..., N).Write an algorithm based on Greedy technique with a Complexity O(nlogn).

Input Format:

First line specifies the number of elements-n

The next n lines contain the array elements.

Output Format:

Maximum Array Sum to be printed.

Sample Input:

5

2 5 3 4 0

Sample output:

40

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>
int cmp(const void *a, const void *b)
{
    return (*(long long *)a - *(long long *)b);
}
int main()
{
    int n;
    scanf("%d", &n);

    long long arr[n];

    for(int i = 0; i < n; i++)
    {
        scanf("%lld", &arr[i]);
    }

    qsort(arr, n, sizeof(long long), cmp);

    long long max_sum = 0;

    for(int i = 0; i < n; i++)
    {
        max_sum += arr[i] * i;
    }
    printf("%lld", max_sum);
    return 0;
}
```

	Input	Expected	Got	
✔	5 2 5 3 4 0	40	40	✔
✔	10 2 2 2 4 4 3 3 5 5 5	191	191	✔
✔	2 45 3	45	45	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Finish review

Back to Course

5-G-Product of Array elements-Minimum

Started on	Monday, 31 August 2026, 10:30 AM
State	Finished
Completed on	Monday, 31 August 2026, 10:34 AM
Time taken	4 mins 22 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Given two arrays array_One[] and array_Two[] of same size N. We need to first rearrange the arrays such that the sum of the product of pairs(1 element from each) is minimum. That is SUM (A[i] * B[i]) for all i is minimum.

For example:

Input	Result
3 1 2 3 4 5 6	28

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>
int ascending(const void *a, const void *b) {
    return (*(int *)a - *(int *)b);
}
int descending(const void *a, const void *b) {
    return (*(int *)b - *(int *)a);
}
int main() {
    int n;
    scanf("%d", &n);
    int a[n], b[n];
    for (int i = 0; i < n; i++)
        scanf("%d", &a[i]);
    for (int i = 0; i < n; i++)
        scanf("%d", &b[i]);

    qsort(a, n, sizeof(int), ascending);
    qsort(b, n, sizeof(int), descending);
    int sum = 0;
    for (int i = 0; i < n; i++)
        sum += a[i] * b[i];
    printf("%d", sum);
    return 0;
}
```

	Input	Expected	Got	
✔	3 1	28	28	✔

Quiz navigation

1

Finish review

	2 3 4 5 6			
✔	4 7 5 1 2 1 3 4 1	22	22	✔
✔	5 20 10 30 10 40 8 9 4 3 10	590	590	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Finish review

Back to Course

1-Number of Zeros in a Given Array

Started on	Friday, 4 September 2026, 8:34 PM
State	Finished
Completed on	Friday, 4 September 2026, 8:41 PM
Time taken	7 mins
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Problem Statement

Given an array of 1s and 0s this has all 1s first followed by all 0s. Aim is to find the number of 0s. Write a program using Divide and Conquer to Count the number of zeroes in the given array.

Input Format

First Line Contains Integer m – Size of array

Next m lines Contains m numbers – Elements of an array

Output Format

First Line Contains Integer – Number of zeroes present in the given array.

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
int findFirstZero(int arr[], int low, int high, int size) {
    if (high >= low) {
        int mid = low + (high - low) / 2;
        if ((mid == 0 || arr[mid - 1] == 1) && arr[mid] == 0)
            return mid;
        if (arr[mid] == 1)
            return findFirstZero(arr, mid + 1, high, size);
        else
            return findFirstZero(arr, low, mid - 1, size);
    }
    return -1;
}
int main() {
    int m;
    scanf("%d", &m);
    int arr[m];
    for (int i = 0; i < m; i++) {
        scanf("%d", &arr[i]);
    }
    int firstZeroIndex = findFirstZero(arr, 0, m - 1, m);
    int count;
    if (firstZeroIndex == -1)
        count = 0;
    else
        count = m - firstZeroIndex;
    printf("%d\n", count);
    return 0;
}

```

	Input	Expected	Got	

✔	5 1 1 1 0 0	2	2	✔
✔	10 1 1 1 1 1 1 1 1 1 1	0	0	✔
✔	8 0 0 0 0 0 0 0 0 0	8	8	✔
✔	17 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0	2	2	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Finish review

Back to Course

2-Majority Element

Started on	Friday, 4 September 2026, 8:41 PM
State	Finished
Completed on	Friday, 4 September 2026, 8:42 PM
Time taken	58 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Given an array `nums` of size `n`, return *the majority element.*

The majority element is the element that appears more than `⌊n / 2⌋` times. You may assume that the majority element always exists in the array.

Example 1:

Input: nums = [3,2,3]

Output: 3

Example 2:

Input: nums = [2,2,1,1,1,2,2]

Output: 2

Constraints:

- `n == nums.length`
- `1 <= n <= 5 * 10^4`
- `-2^31 <= nums[i] <= 2^31 - 1`

For example:

Input	Result
3 3 2 3	3
7 2 2 1 1 1 2 2	2

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
int majorityElement(int nums[], int n) {
    int candidate = nums[0];
    int count = 1;
    for (int i = 1; i < n; i++) {
        if (nums[i] == candidate) {
            count++;
        } else {
            count--;
            if (count == 0) {
                candidate = nums[i];
                count = 1;
```

Quiz navigation

1

Finish review

```
        }
    }
    return candidate;
}
int main() {
    int n;
    scanf("%d", &n);
    int nums[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &nums[i]);
    }
    int result = majorityElement(nums, n);
    printf("%d\n", result);
    return 0;
}
```

	Input	Expected	Got	
✔	3 3 2 3	3	3	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Finish review

Back to Course

3-Finding Floor Value

Started on	Friday, 4 September 2026, 8:42 PM
State	Finished
Completed on	Friday, 4 September 2026, 8:44 PM
Time taken	2 mins 32 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Problem Statement:

Given a sorted array and a value x, the floor of x is the largest element in array smaller than or equal to x. Write divide and conquer algorithm to find floor of x.

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Value for x

Output Format

First Line Contains Integer – Floor value for x

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
int floorSearch(int arr[], int low, int high, int x) {
    if (low > high)
        return -1;
    if (x >= arr[high])
        return arr[high];
    int mid = low + (high - low) / 2;
    if (arr[mid] == x)
        return arr[mid];
    if (mid > 0 && arr[mid - 1] <= x && x < arr[mid])
        return arr[mid - 1];
    if (x < arr[mid])
        return floorSearch(arr, low, mid - 1, x);
    else
        return floorSearch(arr, mid + 1, high, x);
}
int main() {
    int n, x;
    scanf("%d", &n);
    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    scanf("%d", &x);
    int result = floorSearch(arr, 0, n - 1, x);
    if (result == -1)
        printf("No floor exists\n");
    else
        printf("%d\n", result);
    return 0;
}
```

	Input	Expected	Got	
✔	6 1 2 8 10 12 19 5	2	2	✔
✔	5 10 22 85 108 129 100	85	85	✔
✔	7 3 5 7 9 11 13 15 10	9	9	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Finish review

Back to Course

4-Two Elements sum to x

Started on	Friday, 4 September 2026, 8:45 PM
State	Finished
Completed on	Friday, 4 September 2026, 8:46 PM
Time taken	1 min 2 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Problem Statement:

Given a sorted array of integers say arr[] and a number x. Write a recursive program using divide and conquer strategy to check if there exist two elements in the array whose sum = x. If there exist such two elements then return the numbers, otherwise print as "No".

Note: Write a Divide and Conquer Solution

Input Format

First Line Contains Integer n – Size of array

Next n lines Contains n numbers – Elements of an array

Last Line Contains Integer x – Sum Value

Output Format

First Line Contains Integer – Element1

Second Line Contains Integer – Element2 (Element 1 and Elements 2 together sums to value "x")

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
void findPair(int arr[], int low, int high, int x) {
    if (low >= high) {
        printf("No\n");
        return;
    }
    int sum = arr[low] + arr[high];
    if (sum == x) {
        printf("%d\n%d\n", arr[low], arr[high]);
        return;
    }
    else if (sum > x) {
        findPair(arr, low, high - 1, x);  // move high pointer left
    }
    else {
        findPair(arr, low + 1, high, x);  // move low pointer right
    }
}
int main() {
    int n, x;
    scanf("%d", &n);
    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    scanf("%d", &x);
    findPair(arr, 0, n - 1, x);
    return 0;
}

```

	Input	Expected	Got

✔	4 2 4 8 10 14	4 10	4 10	✔
✔	5 2 4 6 8 10 100	No	No	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Finish review

Back to Course

5-Implementation of Quick Sort

Started on	Friday, 4 September 2026, 8:46 PM
State	Finished
Completed on	Friday, 4 September 2026, 8:47 PM
Time taken	50 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 Correct Mark 1.00 out of 1.00 Flag question

Write a Program to Implement the Quick Sort Algorithm

Input Format:
The first line contains the no of elements in the list-n
The next n lines contain the elements.

Output:
Sorted list of elements

For example:

Input	Result
5 67 34 12 98 78	12 34 67 78 98

Answer:

```c
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++) {
        if (arr[j] <= pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pi = partition(arr, low, high);
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}

int main() {
    int n;
```

```
    scanf("%d", &n);
    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    quickSort(arr, 0, n - 1);

    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");

    return 0;
}
```

	Input	Expected	Got	
✔	5 67 34 12 98 78	12 34 67 78 98	12 34 67 78 98	✔
✔	10 1 56 78 90 32 56 11 10 90 114	1 10 11 32 56 56 78 90 90 114	1 10 11 32 56 56 78 90 90 114	✔
✔	12 9 8 7 6 5 4 3 2 1 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	1 2 3 4 5 6 7 8 9 10 11 90	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Finish review

Back to Course

1-DP-Playing with Numbers

Started on	Tuesday, 29 September 2026, 10:36 PM
State	Finished
Completed on	Tuesday, 29 September 2026, 10:37 PM
Time taken	1 min 17 secs
Grade	**10.00** out of 10.00 (**100**%)

Question 1 Correct Mark 10.00 out of 10.00 Flag question

Playing with Numbers:

Ram and Sita are playing with numbers by giving puzzles to each other. Now it was Ram term, so he gave Sita a positive integer 'n' and two numbers 1 and 3. He asked her to find the possible ways by which the number n can be represented using 1 and 3.Write any efficient algorithm to find the possible ways.

Example 1:

Input: *6*
Output:*6*
Explanation: *There are 6 ways to 6 represent number with 1 and 3*

1+1+1+1+1+1
3+3
1+1+1+3
1+1+3+1
1+3+1+1
3+1+1+1

Input Format

First Line contains the number n

Output Format

Print: The number of possible ways 'n' can be represented using 1 and 3

Sample Input

6

Sample Output

6

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int main() {
    int n;
    scanf("%d", &n);

    // If n is 0, 1, or 2, there is only 1 way (using only 1s)
    if (n == 0 || n == 1 || n == 2) {
        printf("1\n");
        return 0;
    }

    // Using long long to prevent integer overflow for larger values of n
    long long dp[n + 1];
```

Quiz navigation

1

Finish review

```
15
16      // Base cases
17      dp[0] = 1; // 1 way to represent 0 (empty sum)
18      dp[1] = 1; // 1
19      dp[2] = 1; // 1+1
20
21      // Build the DP table bottom-up
22      for (int i = 3; i <= n; i++) {
23          // The number can end with a '+ 1' or a '+ 3'
24          dp[i] = dp[i - 1] + dp[i - 3];
25      }
26
27      // Print the number of ways for n
28      printf("%lld\n", dp[n]);
29
30      return 0;
31  }
```

	Input	Expected	Got	
✔	6	6	6	✔
✔	25	8641	8641	✔
✔	100	24382819596721629	24382819596721629	✔

Passed all tests! ✔

Correct

Marks for this submission: 10.00/10.00.

Finish review

Back to Course

2-DP-Playing with chessboard

Started on	Tuesday, 29 September 2026, 10:38 PM
State	Finished
Completed on	Tuesday, 29 September 2026, 10:39 PM
Time taken	1 min 11 secs
Grade	**10.00** out of 10.00 (**100**%)

Question 1 Correct Mark 10.00 out of 10.00 Flag question

Playing with Chessboard:

Ram is given with an n*n chessboard with each cell with a monetary value. Ram stands at the (0,0), that the position of the top left white rook. He is been given a task to reach the bottom right black rook position (n-1, n-1) constrained that he needs to reach the position by traveling the maximum monetary path under the condition that he can only travel one step right or one step down the board. Help ram to achieve it by providing an efficient DP algorithm.

Example:

Input

3

1 2 4

2 3 4

8 7 1

Output:

19

Explanation:

Totally there will be 6 paths among that the optimal is

Optimal path value:1+2+8+7+1=19

Input Format

First Line contains the integer n

The next n lines contain the n*n chessboard values

Output Format

Print Maximum monetary value of the path

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int n;
    scanf("%d", &n);

    int grid[n][n];
    int dp[n][n];

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            scanf("%d", &grid[i][j]);
        }
```

Quiz navigation

1

Finish review

```c
    }

    dp[0][0] = grid[0][0];

    for (int i = 1; i < n; i++) {
        dp[i][0] = dp[i - 1][0] + grid[i][0];
    }

    for (int j = 1; j < n; j++) {
        dp[0][j] = dp[0][j - 1] + grid[0][j];
    }

    for (int i = 1; i < n; i++) {
        for (int j = 1; j < n; j++) {
            dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]) + grid[i][j];
        }
    }

    printf("%d\n", dp[n - 1][n - 1]);

    return 0;
}
```

	Input	Expected	Got	
✔	3 1 2 4 2 3 4 8 7 1	19	19	✔
✔	3 1 3 1 1 5 1 4 2 1	12	12	✔
✔	4 1 1 3 4 1 5 7 8 2 3 4 6 1 6 9 0	28	28	✔

Passed all tests! ✔

Correct

Marks for this submission: 10.00/10.00.

Finish review

Back to Course

3-DP-Longest Common Subsequence

Started on	Tuesday, 29 September 2026, 10:39 PM
State	Finished
Completed on	Tuesday, 29 September 2026, 10:40 PM
Time taken	37 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Given two strings find the length of the common longest subsequence(need not be contiguous) between the two.

Example:

s1: ggtabe

s2: tgatasb

s1	a	g	**g**	**t**	**a**	**b**	
s2	**g**	x	**t**	x	**a**	y	**b**

The length is 4

Solveing it using Dynamic Programming

For example:

Input	Result
aab azb	2

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <string.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    char s1[1005];
    char s2[1005];

    scanf("%s", s1);
    scanf("%s", s2);

    int m = strlen(s1);
    int n = strlen(s2);

    int dp[m + 1][n + 1];

    for (int i = 0; i <= m; i++) {
        for (int j = 0; j <= n; j++) {
```

```
            if (i == 0 || j == 0) {
                dp[i][j] = 0;
            } else if (s1[i - 1] == s2[j - 1]) {
                dp[i][j] = dp[i - 1][j - 1] + 1;
            } else {
                dp[i][j] = max(dp[i - 1][j], dp[i][j - 1]);
            }
        }
    }

    printf("%d\n", dp[m][n]);

    return 0;
}
```

	Input	Expected	Got	
✔	aab azb	2	2	✔
✔	ABCD ABCD	4	4	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Finish review

Back to Course

4-DP-Longest non-decreasing Subsequence

Started on	Tuesday, 29 September 2026, 10:40 PM
State	Finished
Completed on	Tuesday, 29 September 2026, 10:41 PM
Time taken	44 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Problem statement:

Find the length of the Longest Non-decreasing Subsequence in a given Sequence.

Eg:

Input:9

Sequence:[-1,3,4,5,2,2,2,2,3]

the subsequence is [-1,2,2,2,2,3]

Output:6

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) {
        return 0;
    }

    if (n == 0) {
        printf("0\n");
        return 0;
    }

    int arr[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    int dp[n];
    int max_len = 0;

    for (int i = 0; i < n; i++) {
        dp[i] = 1;
        for (int j = 0; j < i; j++) {
            if (arr[i] >= arr[j] && dp[i] < dp[j] + 1) {
                dp[i] = dp[j] + 1;
            }
        }
        if (dp[i] > max_len) {
            max_len = dp[i];
        }
    }

    printf("%d\n", max_len);

    return 0;
}
```

Quiz navigation

1

Finish review

	Input	Expected	Got	
✔	9 -1 3 4 5 2 2 2 2 3	6	6	✔
✔	7 1 2 2 4 5 7 6	6	6	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Finish review

Back to Course

1-Finding Duplicates-O(n^2) Time Complexity,O(1) Space Complexity

Started on	Saturday, 10 October 2026, 10:32 PM
State	Finished
Completed on	Saturday, 10 October 2026, 10:34 PM
Time taken	1 min 20 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Find Duplicate in Array.

Given a read only array of n integers between 1 and n, find one number that repeats.

Input Format:

First Line - Number of elements

n Lines - n Elements

Output Format:

Element x - That is repeated

For example:

Input	Result
5 1 1 2 3 4	1

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    // Read the number of elements
    if (scanf("%d", &n) != 1) return 0;

    // Create a frequency array initialized to 0 to track numbers from 1 to n
    int *freq = (int *)calloc(n + 1, sizeof(int));
    int duplicate = -1;

    // Read elements one by one and check for duplicates
    for (int i = 0; i < n; i++) {
        int num;
        scanf("%d", &num);
        freq[num]++;
        if (freq[num] > 1) {
            duplicate = num;
        }
    }

    // Print the repeated element
    printf("%d\n", duplicate);

    free(freq);
    return 0;
}
```

	Input	Expected	Got	
✔	11 10 9 7 6 5 1 2 3 8 4 7	7	7	✔
✔	5 1 2 3 4 4	4	4	✔
✔	5 1 1 2 3 4	1	1	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Finish review

Back to Course

2-Finding Duplicates-O(n) Time Complexity,O(1) Space Complexity

Started on	Saturday, 10 October 2026, 10:34 PM
State	Finished
Completed on	Saturday, 10 October 2026, 10:34 PM
Time taken	30 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Find Duplicate in Array.

Given a read only array of n integers between 1 and n, find one number that repeats.

Input Format:

First Line - Number of elements

n Lines - n Elements

Output Format:

Element x - That is repeated

For example:

Input	Result
5 1 1 2 3 4	1

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    // Read the number of elements
    if (scanf("%d", &n) != 1) return 0;

    // Create a frequency array to track numbers from 1 to n
    int *freq = (int *)calloc(n + 1, sizeof(int));
    int duplicate = -1;

    // Read the array elements and find the duplicate
    for (int i = 0; i < n; i++) {
        int num;
        scanf("%d", &num);
        freq[num]++;
        if (freq[num] > 1) {
            duplicate = num;
        }
    }

    // Print the repeated element
    printf("%d\n", duplicate);

    free(freq);
    return 0;
}
```

	Input	Expected	Got	
✔	11 10 9 7 6 5 1 2 3 8 4 7	7	7	✔
✔	5 1 2 3 4 4	4	4	✔
✔	5 1 1 2 3 4	1	1	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Finish review

Back to Course

3-Print Intersection of 2 sorted arrays-O(m*n)Time Complexity,O(1) Space Complexity

Started on	Saturday, 10 October 2026, 10:34 PM
State	Finished
Completed on	Saturday, 10 October 2026, 10:36 PM
Time taken	1 min 3 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format

· The first line contains T, the number of test cases. Following T lines contain:

1. Line 1 contains N1, followed by N1 integers of the first array
2. Line 2 contains N2, followed by N2 integers of the second array

Output Format

The intersection of the arrays in a single line

Example

Input:

1

3 10 17 57

6 2 7 10 15 57 246

Output:

10 57

Input:

1

6 1 2 3 4 5 6

2 1 6

Output:

1 6

For example:

Input	Result
1 3 10 17 57 6 2 7 10 15 57 246	10 57

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

void solve() {
    int n1;
    if (scanf("%d", &n1) != 1) return;

    int *arr1 = (int *)malloc(n1 * sizeof(int));
    for (int i = 0; i < n1; i++) {
        scanf("%d", &arr1[i]);
    }

    int n2;
    scanf("%d", &n2);

    int *arr2 = (int *)malloc(n2 * sizeof(int));
    for (int j = 0; j < n2; j++) {
        scanf("%d", &arr2[j]);
    }

    int i = 0, j = 0;
    int first = 1; // Flag to handle spacing correctly for output formatting

    while (i < n1 && j < n2) {
        if (arr1[i] == arr2[j]) {
            if (!first) {
                printf(" ");
            }
            printf("%d", arr1[i]);
            first = 0;
            i++;
            j++;
        } else if (arr1[i] < arr2[j]) {
            i++;
        } else {
            j++;
        }
    }
    printf("\n");

    free(arr1);
    free(arr2);
}

int main() {
    int t;
    if (scanf("%d", &t) == 1) {
        while (t--) {
            solve();
        }
    }
    return 0;
```

	Input	Expected	Got	
✔	1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57	✔
✔	1 6 1 2 3 4 5 6 2 1 6	1 6	1 6	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Finish review

Back to Course

4-Print Intersection of 2 sorted arrays-O(m+n)Time Complexity,O(1) Space Complexity

Started on	Saturday, 10 October 2026, 10:36 PM
State	Finished
Completed on	Saturday, 10 October 2026, 10:37 PM
Time taken	1 min 18 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Find the intersection of two sorted arrays.

OR in other words,

Given 2 sorted arrays, find all the elements which occur in both the arrays.

Input Format

· The first line contains T, the number of test cases. Following T lines contain:

1. Line 1 contains N1, followed by N1 integers of the first array
2. Line 2 contains N2, followed by N2 integers of the second array

Output Format

The intersection of the arrays in a single line

Example

Input:

1

3 10 17 57

6 2 7 10 15 57 246

Output:

10 57

Input:

1

6 1 2 3 4 5 6

2 1 6

Output:

1 6

For example:

Input	Result
1 3 10 17 57 6 2 7 10 15 57 246	10 57

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

void solve() {
    int n1;
    if (scanf("%d", &n1) != 1) return;

    int *arr1 = (int *)malloc(n1 * sizeof(int));
    for (int i = 0; i < n1; i++) {
        scanf("%d", &arr1[i]);
    }

    int n2;
    scanf("%d", &n2);

    int *arr2 = (int *)malloc(n2 * sizeof(int));
    for (int j = 0; j < n2; j++) {
        scanf("%d", &arr2[j]);
    }

    int i = 0, j = 0;
    int first = 1; // Used for correct space formatting between output numbers

    while (i < n1 && j < n2) {
        if (arr1[i] == arr2[j]) {
            if (!first) {
                printf(" ");
            }
            printf("%d", arr1[i]);
            first = 0;
            i++;
            j++;
        } else if (arr1[i] < arr2[j]) {
            i++;
        } else {
            j++;
        }
    }
    printf("\n");

    free(arr1);
    free(arr2);
}

int main() {
    int t;
    if (scanf("%d", &t) == 1) {
        while (t--) {
            solve();
        }
    }
    return 0;
```

	Input	Expected	Got	
✔	1 3 10 17 57 6 2 7 10 15 57 246	10 57	10 57	✔
✔	1 6 1 2 3 4 5 6 2 1 6	1 6	1 6	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Finish review

Back to Course

5-Pair with Difference-O(n^2)Time Complexity,O(1) Space Complexity

Started on	Saturday, 10 October 2026, 10:37 PM
State	Finished
Completed on	Saturday, 10 October 2026, 10:38 PM
Time taken	29 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that A[j] - A[i] = k, i != j.

Input Format:

First Line n - Number of elements in an array

Next n Lines - N elements in the array

k - Non - Negative Integer

Output Format:

1 - If pair exists

0 - If no pair exists

Explanation for the given Sample Testcase:

YES as 5 - 1 = 4

So Return 1.

For example:

Input	Result
3 1 3 5 4	1

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int *A = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        scanf("%d", &A[i]);
    }

    int k;
    scanf("%d", &k);

    // Handle negative k by taking absolute value if needed,
    // but the problem states k is non-negative.
    int found = 0;
    int i = 0, j = 1;
```

```c
    while (i < n && j < n) {
        if (i != j && (A[j] - A[i] == k)) {
            found = 1;
            break;
        } else if (A[j] - A[i] < k) {
            j++;
        } else {
            i++;
        }
        if (i == j) {
            j++;
        }
    }

    printf("%d\n", found);

    free(A);
    return 0;
}
```

	Input	Expected	Got	
✔	3 1 3 5 4	1	1	✔
✔	10 1 4 6 8 12 14 15 20 21 25 1	1	1	✔
✔	10 1 2 3 5 11 14 16 24 28 29 0	0	0	✔
✔	10 0 2 3 7 13 14 15 20 24 25 10	1	1	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Finish review

Back to Course

6-Pair with Difference -O(n) Time Complexity,O(1) Space Complexity

Started on	Saturday, 10 October 2026, 10:38 PM
State	Finished
Completed on	Saturday, 10 October 2026, 10:38 PM
Time taken	28 secs
Marks	1.00/1.00
Grade	**10.00** out of 10.00 (**100**%)

Question 1 | Correct Mark 1.00 out of 1.00 Flag question

Given an array A of sorted integers and another non negative integer k, find if there exists 2 indices i and j such that A[j] - A[i] = k, i != j.

Input Format:

First Line n - Number of elements in an array

Next n Lines - N elements in the array

k - Non - Negative Integer

Output Format:

1 - If pair exists

0 - If no pair exists

Explanation for the given Sample Testcase:

YES as 5 - 1 = 4

So Return 1.

For example:

Input	Result
3 1 3 5 4	1

Answer: (penalty regime: 0 %)

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int *A = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        scanf("%d", &A[i]);
    }

    int k;
    scanf("%d", &k);

    int found = 0;
    int i = 0, j = 1;

    while (i < n && j < n) {
```

```
        int diff = A[j] - A[i];
        if (i != j && diff == k) {
            found = 1;
            break;
        } else if (diff < k) {
            j++;
        } else {
            i++;
        }
        if (i == j) {
            j++;
        }
    }

    printf("%d\n", found);

    free(A);
    return 0;
}
```

	Input	Expected	Got	
✔	3 1 3 5 4	1	1	✔
✔	10 1 4 6 8 12 14 15 20 21 25 1	1	1	✔
✔	10 1 2 3 5 11 14 16 24 28 29 0	0	0	✔
✔	10 0 2 3 7 13 14 15 20 24 25 10	1	1	✔

Passed all tests! ✔

Correct

Marks for this submission: 1.00/1.00.

Finish review

Back to Course